

Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 9

по дисциплине «Объектно-ориентированное программирование»
на тему «Сетевое программирование. Доска объявлений на Windows Forms»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ОПИСАНИЕ ПРИЛОЖЕНИЯ

Приложение представляет собой децентрализованную «Доску объявлений», работающую в локальной сети. Оно позволяет пользователям подключаться к общей группе, публиковать свои объявления, видеть объявления других пользователей в реальном времени, а также отправлять личные сообщения продавцам.

Ключевые технологии:

- **Язык:** C#
- **UI Фреймворк:** Avalonia UI (MVVM-lite / Code-Behind)
- **Сетевое взаимодействие:** UDP Multicast (для общей рассылки) и UDP Unicast (для личных сообщений)

2. ОСНОВНЫЕ КОМПОНЕНТЫ (АРХИТЕКТУРА И КОД)

2.1 Сетевой сервис: MulticastService.cs

Это ядро приложения, отвечающее за отправку и приём пакетов.

Ключевые особенности:

- Multicast-группа:** Используется IP-адрес 235.5.5.1 и порт 8001. Все пользователи, подключённые к этой группе, получают широковещательные сообщения.
- Формат сообщений:** Сообщения передаются в виде строк, разделённых символом «|». Например:
AD|Имя|Заголовок|Цена|ПортДляСвязи.
- Асинхронность:** Приём сообщений работает в отдельных фоновых задачах (Task.Run), чтобы не блокировать UI.
- Личные сообщения (Unicast):** При публикации объявления пользователь передаёт свой уникальный порт (ChatPort). Когда кто-то хочет написать личное сообщение, оно отправляется напрямую на IP-адрес отправителя и этот порт.

Полный код: MulticastService.cs

```
using System;
```

```

using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace AdsBoardApp.Services;

public class MulticastService
{
    private const string HOST = "235.5.5.1";
    private const int PORT = 8001;
    private const int TTL = 20;

    private readonly IPAddress groupAddress = IPAddress.Parse(HOST);
    private readonly ConcurrentDictionary<string, (IPAddress Ip, int Port)>
sellerContacts = new();

    private UdpClient? client;
    private UdpClient? chatClient;
    private Task? receiveTask;
    private Task? chatReceiveTask;
    private CancellationTokenSource? cts;
    private volatile bool alive;
    private string userName = string.Empty;

    public int ChatPort { get; private set; }
    public bool IsOnline => alive;
    public string UserName => userName;

    public event Action<string, string, string, string>? AdReceived;
    public event Action<string>? SystemReceived;
    public event Action<string, string>? PrivateMessageReceived;

    public void Login(string name)
    {
        if (alive) return;
        userName = name;
        Socket socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram,
ProtocolType.Udp);
        socket.SetSocketOption(SocketOptionLevel.Socket,
SocketOptionName.ReuseAddress, true);
        socket.Bind(new IPEndPoint(IPAddress.Any, PORT));
        client = new UdpClient { Client = socket };
        client.JoinMulticastGroup(groupAddress, TTL);
        chatClient = new UdpClient(0);
        ChatPort = ((IPEndPoint)chatClient.Client.LocalEndPoint!).Port;
        alive = true;
        cts = new CancellationTokenSource();
        receiveTask = Task.Run(ReceiveLoopAsync);
        chatReceiveTask = Task.Run(ChatReceiveLoopAsync);
        SendSystem($"{userName} вошёл на доску объявлений");
    }

    public void Logout()

```

```

{
    if (!alive) return;
    SendSystem($"{userName} покинул доску объявлений");
    alive = false;
    cts?.Cancel();
    try { client?.DropMulticastGroup(groupAddress); } catch { }
    try { receiveTask?.Wait(TimeSpan.FromSeconds(2)); } catch { }
    try { chatReceiveTask?.Wait(TimeSpan.FromSeconds(2)); } catch { }
    client?.Close();
    chatClient?.Close();
    client = null;
    chatClient = null;
    receiveTask = null;
    chatReceiveTask = null;
    cts?.Dispose();
    cts = null;
    sellerContacts.Clear();
}

public void PublishAd(string title, string price)
{
    if (!alive || client == null) return;
    string packed = string.Join("|", "AD", userName, title, price,
ChatPort.ToString());
    byte[] data = Encoding.Unicode.GetBytes(packed);
    client.Send(data, data.Length, HOST, PORT);
}

public bool SendPrivateMessage(string sellerName, string text)
{
    if (!sellerContacts.TryGetValue(sellerName, out var contact)) return false;
    using UdpClient directSender = new UdpClient();
    string packed = string.Join("|", "MSG", userName, text);
    byte[] data = Encoding.Unicode.GetBytes(packed);
    directSender.Send(data, data.Length, contact.Ip.ToString(), contact.Port);
    return true;
}

private void SendSystem(string text)
{
    if (client == null) return;
    string packed = string.Join("|", "SYS", text);
    byte[] data = Encoding.Unicode.GetBytes(packed);
    client.Send(data, data.Length, HOST, PORT);
}

private async Task ReceiveLoopAsync()
{
    UdpClient localClient = client!;
    try
    {
        while (alive)
        {
            var result = await localClient.ReceiveAsync(cts!.Token);
            byte[] data = result.Buffer;
            IPEndPoint remoteIp = result.RemoteEndPoint;
            string packed = Encoding.Unicode.GetString(data);
            string[] parts = packed.Split('|');
            if (parts.Length == 0) continue;

```

```

        switch (parts[0])
        {
            case "AD":
                if (parts.Length >= 5)
                {
                    string seller = parts[1];
                    string title = parts[2];
                    string price = parts[3];
                    if (int.TryParse(parts[4], out int sellerChatPort) &&
remoteIp != null)
                        sellerContacts[seller] = (remoteIp.Address,
sellerChatPort);

                    string time = DateTime.Now.ToString("HH:mm");
                    AdReceived?.Invoke(time, seller, title, price);
                }
                break;
            case "SYS":
                if (parts.Length >= 2)
                    SystemReceived?.Invoke(parts[1]);
                break;
        }
    }
}
catch (OperationCanceledException) { }
catch (ObjectDisposedException) { if (!alive) return; throw; }
catch (SocketException) { if (!alive) return; throw; }
}

private async Task ChatReceiveLoopAsync()
{
    UdpClient localChatClient = chatClient!;
    try
    {
        while (alive)
        {
            var result = await localChatClient.ReceiveAsync(cts!.Token);
            byte[] data = result.Buffer;
            string packed = Encoding.Unicode.GetString(data);
            string[] parts = packed.Split('|');
            if (parts.Length >= 3 && parts[0] == "MSG")
                PrivateMessageReceived?.Invoke(parts[1], parts[2]);
        }
    }
    catch (OperationCanceledException) { }
    catch (ObjectDisposedException) { if (!alive) return; throw; }
    catch (SocketException) { if (!alive) return; throw; }
}
}

```

2.2 Модель данных: AdItem.cs

Простая модель для отображения элемента в списке объявлений.

```

namespace AdsBoardApp.Models;

public class AdItem
{
    public string Display { get; set; } = string.Empty;
    public string? Seller { get; set; }
}

```

2.3 Главное окно (UI): MainWindow.axaml и MainWindow.axaml.cs

Интерфейс построен на Avalonia UI. В MainWindow.axaml.cs происходит подписка на события от MulticastService.

Важный аспект: так как события от сети приходят в фоновых потоках, обновление UI (добавление элементов в ObservableCollection) обязательно должно происходить в главном потоке интерфейса (UI Thread). Для этого используется Dispatcher.UIThread.Post.

Полный код: MainWindow.axaml

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:models="clr-namespace:AdsBoardApp.Models"
        x:Class="AdsBoardApp.Views.MainWindow"
        Title="Доска объявлений — Лабораторная работа №9"
        Width="800" Height="700">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!-- Панель входа -->
        <Border Grid.Row="0" BorderBrush="Gray" BorderThickness="1"
                Padding="10" Margin="0,0,0,10">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <TextBlock Text="Имя пользователя:" VerticalAlignment="Center"
                           FontWeight="Bold"/>
                <TextBox Name="UserNameTextBox" Width="200"
                         PlaceholderText="Введите имя"/>
                <Button Name="LoginButton" Content="Войти"
                       Click="OnLogin" Width="100" Height="32"/>
                <Button Name="LogoutButton" Content="Выйти"
                       Click="OnLogout" Width="100" Height="32"
                       IsEnabled="False"/>
                <TextBlock Name="StatusTextBlock" VerticalAlignment="Center"
                           FontStyle="Italic" Foreground="Gray"
                           Margin="10,0,0,0" Text="Не подключено"/>
            </StackPanel>
        </Border>

        <!-- Новое объявление -->
        <Border Grid.Row="1" BorderBrush="Gray" BorderThickness="1"
                Padding="10" Margin="0,0,0,10">
            <StackPanel Spacing="8">
                <TextBlock Text="Новое объявление" FontWeight="Bold"/>
                <StackPanel Orientation="Horizontal" Spacing="10">
                    <TextBlock Text="Заголовок:" VerticalAlignment="Center"
                               Width="80"/>
```

```

        <TextBox Name="TitleTextBox" Width="300"
                PlaceholderText="Например, Велосипед"/>
    </StackPanel>
    <StackPanel Orientation="Horizontal" Spacing="10">
        <TextBlock Text="Цена (₽):" VerticalAlignment="Center"
Width="80"/>
        <TextBox Name="PriceTextBox" Width="150"
PlaceholderText="3000"/>
        <Button Name="PublishButton" Content="Опубликовать"
                Click="OnPublish" Width="140" Height="32"
                IsEnabled="False"/>
    </StackPanel>
</StackPanel>
</Border>

<TextBlock Grid.Row="2"
            Text="Лента объявлений и уведомлений:"
            FontWeight="Bold" Margin="0,0,0,5"/>

<Border Grid.Row="3" BorderBrush="Gray" BorderThickness="1">
    <ListBox Name="AdsListBox" Padding="5"
            SelectionChanged="OnAdSelectionChanged">
        <ListBox.ItemTemplate>
            <DataTemplate DataType="models:AdItem">
                <TextBlock Text="{Binding Display}"
                        TextWrapping="Wrap" Padding="4"/>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>
</Border>

<StackPanel Grid.Row="4" Orientation="Horizontal"
            Spacing="10" Margin="0,10,0,0">
    <Button Name="WriteSellerButton" Content="Написать продавцу"
            Click="OnWriteSeller" Width="180" Height="32"
            IsEnabled="False"/>
    <TextBlock VerticalAlignment="Center" FontStyle="Italic"
            Foreground="Gray"
            Text="Выберите объявление, чтобы написать продавцу
(unicast)"/>
</StackPanel>
</Grid>
</Window>

```

Полный код: MainWindow.axaml.cs

```

using System;
using System.Collections.ObjectModel;
using Avalonia.Controls;
using Avalonia.Interactivity;
using Avalonia.Threading;
using AdsBoardApp.Models;
using AdsBoardApp.Services;

namespace AdsBoardApp.Views;

public partial class MainWindow : Window
{

```

```

private readonly MulticastService service = new();
private readonly ObservableCollection<AdItem> ads = new();

public MainWindow()
{
    InitializeComponent();
    AdsListBox.ItemsSource = ads;
    service.AdReceived += OnAdReceived;
    service.SystemReceived += OnSystemReceived;
    service.PrivateMessageReceived += OnPrivateMessageReceived;
    Closing += OnWindowClosing;
}

private void OnLogin(object? sender, RoutedEventArgs e)
{
    string name = (UserNameTextBox.Text ?? string.Empty).Trim();
    if (string.IsNullOrEmpty(name))
    {
        StatusTextBlock.Text = "Введите имя пользователя";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
        return;
    }
    try
    {
        service.Login(name);
        UserNameTextBox.IsEnabled = false;
        LoginButton.IsEnabled = false;
        LogoutButton.IsEnabled = true;
        PublishButton.IsEnabled = true;
        StatusTextBlock.Text = $"В сети как «{name}»";
(chatPort={service.ChatPort})";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Green;
    }
    catch (Exception ex)
    {
        StatusTextBlock.Text = $"Ошибка входа: {ex.Message}";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
    }
}

private void OnLogout(object? sender, RoutedEventArgs e)
{
    service.Logout();
    ResetUiToOffline();
}

private void ResetUiToOffline()
{
    UserNameTextBox.IsEnabled = true;
    LoginButton.IsEnabled = true;
    LogoutButton.IsEnabled = false;
    PublishButton.IsEnabled = false;
    WriteSellerButton.IsEnabled = false;
    StatusTextBlock.Text = "Не подключено";
    StatusTextBlock.Foreground = Avalonia.Media.Brushes.Gray;
}

private void OnPublish(object? sender, RoutedEventArgs e)
{

```



```

        string title = (TitleTextBox.Text ?? string.Empty).Trim();
        string price = (PriceTextBox.Text ?? string.Empty).Trim();
        if (string.IsNullOrEmpty(title) || string.IsNullOrEmpty(price))
return;
        service.PublishAd(title, price);
        TitleTextBox.Text = string.Empty;
        PriceTextBox.Text = string.Empty;
    }

    private void OnAdReceived(string time, string seller, string title, string
price)
    {
        Dispatcher.UIThread.Post(() =>
        {
            ads.Insert(0, new AdItem
            {
                Display = $"[{time}] {seller}: {title} - {price} py6.",
                Seller = seller,
            });
        });
    }

    private void OnSystemReceived(string text)
    {
        Dispatcher.UIThread.Post(() =>
        {
            ads.Insert(0, new AdItem { Display = $"→ {text}", Seller = null });
        });
    }

    private void OnPrivateMessageReceived(string sender, string text)
    {
        Dispatcher.UIThread.Post(async () =>
        {
            await ShowInfo($"Сообщение от {sender}", text);
        });
    }

    private void OnAdSelectionChanged(object? sender, SelectionChangedEventArgs e)
    {
        WriteSellerButton.IsEnabled =
            service.IsOnline
            && AdsListBox.SelectedItem is AdItem item
            && !string.IsNullOrEmpty(item.Seller)
            && item.Seller != service.UserName;
    }

    private async void OnWriteSeller(object? sender, RoutedEventArgs e)
    {
        if (AdsListBox.SelectedItem is not AdItem item
            || string.IsNullOrEmpty(item.Seller)) return;

        var dialog = new WriteToSellerWindow(item.Seller);
        var text = await dialog.ShowDialog<string?>(this);
        if (string.IsNullOrEmpty(text)) return;

        bool sent = service.SendPrivateMessage(item.Seller, text);
        if (!sent)

```

```

        await ShowInfo("Не удалось отправить",
            $"Контакт продавца «{item.Seller}» не найден. " +
            "Дождитесь его следующего объявления.");
    }

    private void OnWindowClosing(object? sender, WindowClosingEventArgs e)
    {
        if (service.IsOnline) service.Logout();
    }

    private async System.Threading.Tasks.Task ShowInfo(string title, string
message)
    {
        var win = new Window
        {
            Title = title, Width = 420, Height = 200,
            WindowStartupLocation = WindowStartupLocation.CenterOwner,
            CanResize = false,
        };
        var panel = new StackPanel { Margin = new Avalonia.Thickness(20), Spacing =
15 };
        panel.Children.Add(new TextBlock
            { Text = message, FontSize = 14,
              TextWrapping = Avalonia.Media.TextWrapping.Wrap });
        var btn = new Button
            { Content = "OK", Width = 100, Height = 32,
              HorizontalAlignment = Avalonia.Layout.HorizontalAlignment.Center };
        btn.Click += (_, _) => win.Close();
        panel.Children.Add(btn);
        win.Content = panel;
        await win.ShowDialog(this);
    }
}

```

2.4 Окно личных сообщений: WriteToSellerWindow

Вспомогательное окно для ввода текста личного сообщения продавцу.

```

<!-- WriteToSellerWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    x:Class="AdsBoardApp.Views.WriteToSellerWindow"
    Title="Личное сообщение"
    Width="450" Height="280"
    WindowStartupLocation="CenterOwner"
    CanResize="False">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <TextBlock Grid.Row="0" Name="HeaderTextBlock"
            FontWeight="Bold" FontSize="14" Margin="0,0,0,10"/>
        <TextBox Grid.Row="1" Name="MessageTextBox"
            AcceptsReturn="True" TextWrapping="Wrap"
            PlaceholderText="Введите сообщение продавцу..."
            VerticalContentAlignment="Top"/>
        <StackPanel Grid.Row="2" Orientation="Horizontal"
            HorizontalAlignment="Right" Spacing="10" Margin="0,10,0,0">

```

```

        <Button Content="Отмена"      Width="100" Height="32" Click="OnCancel"/>
        <Button Content="Отправить" Width="120" Height="32"
            Click="OnSend" IsDefault="True"/>
    </StackPanel>
</Grid>
</Window>

```

```

// WriteToSellerWindow.axaml.cs
using Avalonia.Controls;
using Avalonia.Interactivity;

namespace AdsBoardApp.Views;

public partial class WriteToSellerWindow : Window
{
    public WriteToSellerWindow() { InitializeComponent(); }

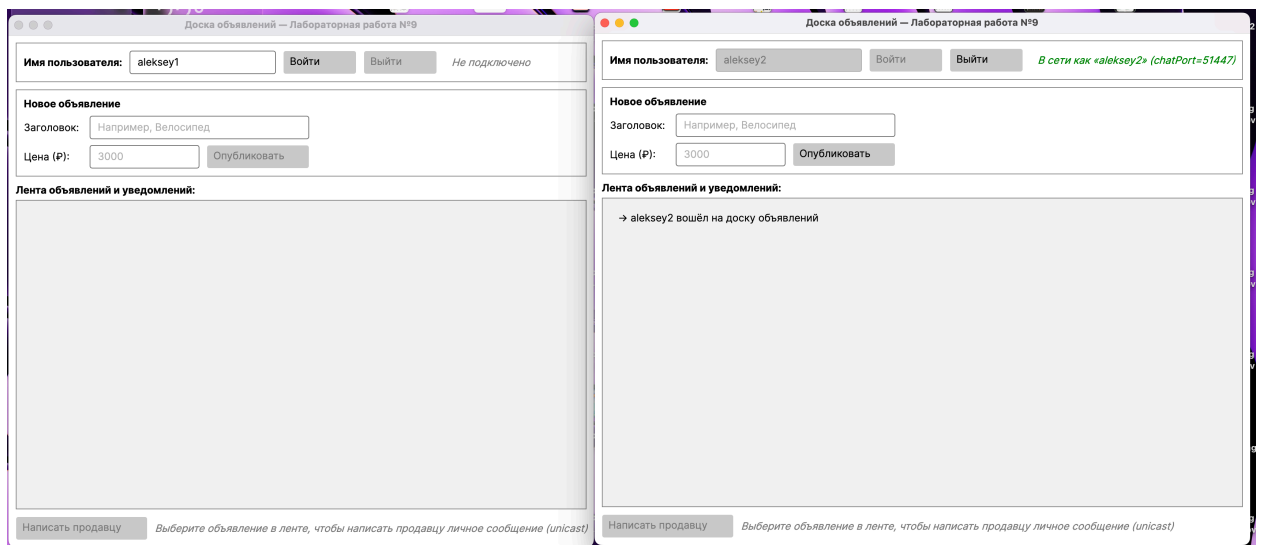
    public WriteToSellerWindow(string sellerName) : this()
    {
        Title = $"Сообщение для {sellerName}";
        HeaderTextBlock.Text = $"Личное сообщение продавцу: {sellerName}";
    }

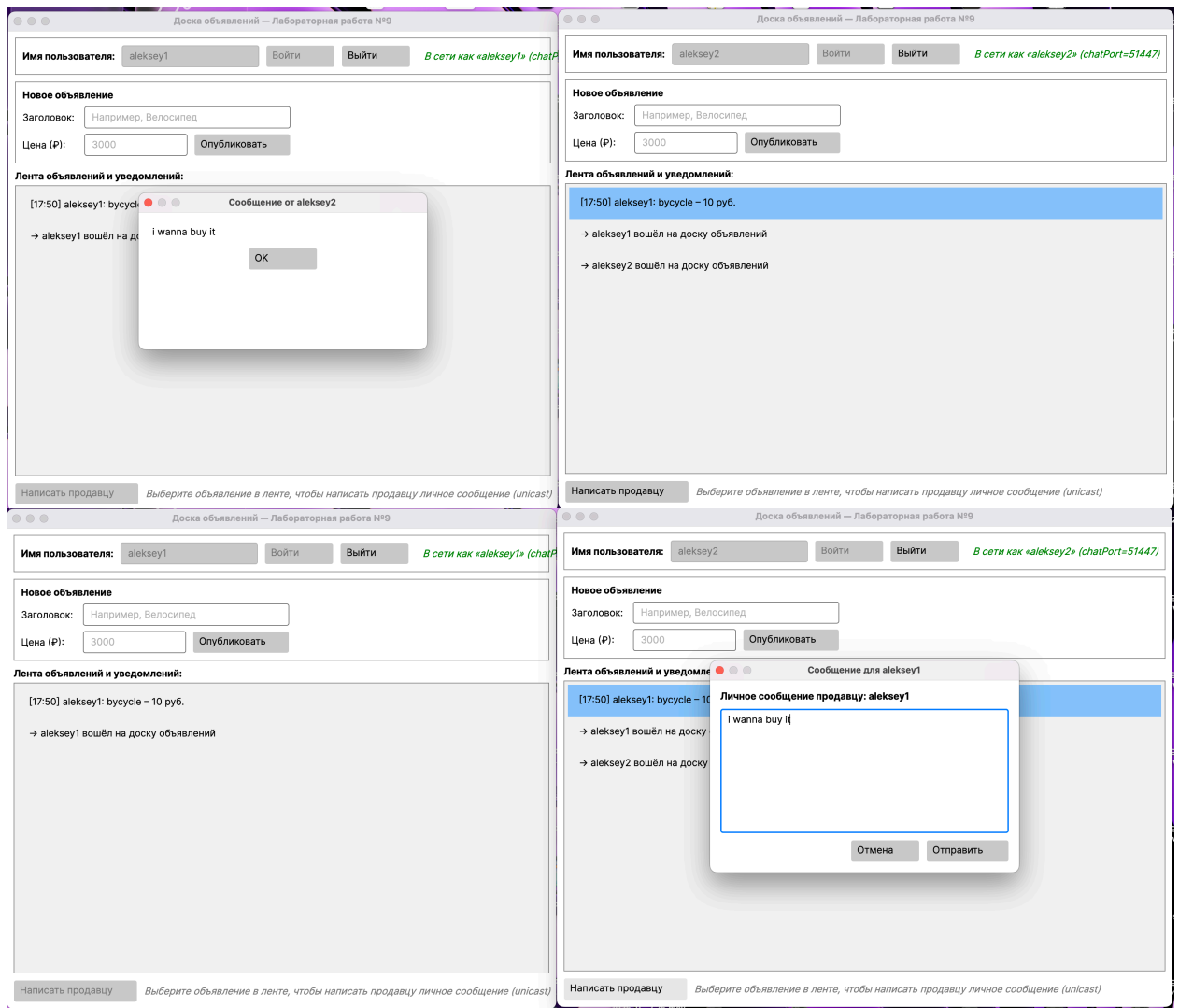
    private void OnSend(object? sender, RoutedEventArgs e) =>
        Close(MessageTextBox.Text);
    private void OnCancel(object? sender, RoutedEventArgs e) => Close(null);
}

```

3. СКРИНШОТЫ РАБОТЫ ПРИЛОЖЕНИЯ

Ниже представлены скриншоты, демонстрирующие работу приложения.





4. ВЫВОД

В ходе выполнения лабораторной работы было разработано приложение «Доска объявлений», функционирующее в локальной сети на основе протокола UDP Multicast. Реализован полный цикл сетевого взаимодействия: подключение к Multicast-группе, публикация и приём объявлений в режиме реального времени, а также отправка личных сообщений между пользователями через UDP Unicast.

В процессе работы были закреплены навыки асинхронного программирования на C# с использованием `async/await` и `CancellationToken`, что позволило корректно управлять жизненным циклом фоновых задач и избежать блокировок UI-потoka. Отдельное внимание было уделено межпоточному взаимодействию: обновление элементов интерфейса из фоновых потоков выполняется через `Dispatcher.UIThread.Post`, что соответствует архитектурным требованиям платформы Avalonia UI.

Были изучены ключевые отличия между Multicast и Broadcast, принципы работы IGMP-маршрутизации, а также механизм TTL для

управления областью распространения пакетов. Полученные знания и практический опыт разработки сетевых приложений могут быть применены при создании распределённых систем реального времени.